FAST National University of Computer and Emerging Sciences

Islamabad Campus

Department of Computer Sciences

AI4015 Agentic Artificial Intelligence, Fall 2026

Semester Project, Phase 1: Literature Review, Proposal and Baseline

---

Verified Optimization Agents

Requirement-Level Verification and Calibrated Abstention for LLM-Based Optimization Modeling

---

Selected seed (G1-01): LLM Agents that Translate Natural Language into Verified Optimization Models

Name	Roll Number	Role
Mushahid Hussain	23I-0541	Group head
Muhammad Waqas	23I-0569	Member
Abdul Rauf	23I-0591	Member

Supervisor: Muhammad Sajjad (G1 projects)
Instructor: Muhammad Owais Idrees
Repository: https://github.com/mushi0541/verified-optimization-agents
Submitted: 20 October 2026

Contents

1 Literature Review and Research Gaps **1**
1.1 Base Paper Selection and Validation . . . 2
1.2 Critical Summaries of Nine Additional Papers . . . 4
1.3 Cross-Paper Comparison . . . 8
1.4 Research Gap Synthesis . . . 8

2 Project Proposal **12**
2.1 Project Title and Selected Seed . . . 12
2.2 Problem Statement and Motivation . . . 12
2.3 Base Paper and Research Gaps Summary . . . 13
2.4 Target Product or System . . . 13
2.5 Agentic Justification . . . 14
2.6 Goals and Success Criteria . . . 14
2.7 Data / Environment / Source . . . 15
2.8 System Architecture Description . . . 15
2.9 Methodology and Architecture Blueprint . . . 17
2.10 Baseline Definition . . . 17
2.11 Research Questions and Hypotheses . . . 18
2.12 Evaluation Plan . . . 18
2.13 Milestones and Feasibility . . . 19
2.14 Work Division Matrix . . . 20
2.15 Potential Publication Direction . . . 21

3 Technical Foundation and Initial Results **21**
3.1 Phase 1 Technical Requirements . . . 21
3.2 Initial Baseline Results . . . 21
3.3 First End-to-End Run of the Proposed Pipeline . . . 22

References **24**

1 Literature Review and Research Gaps

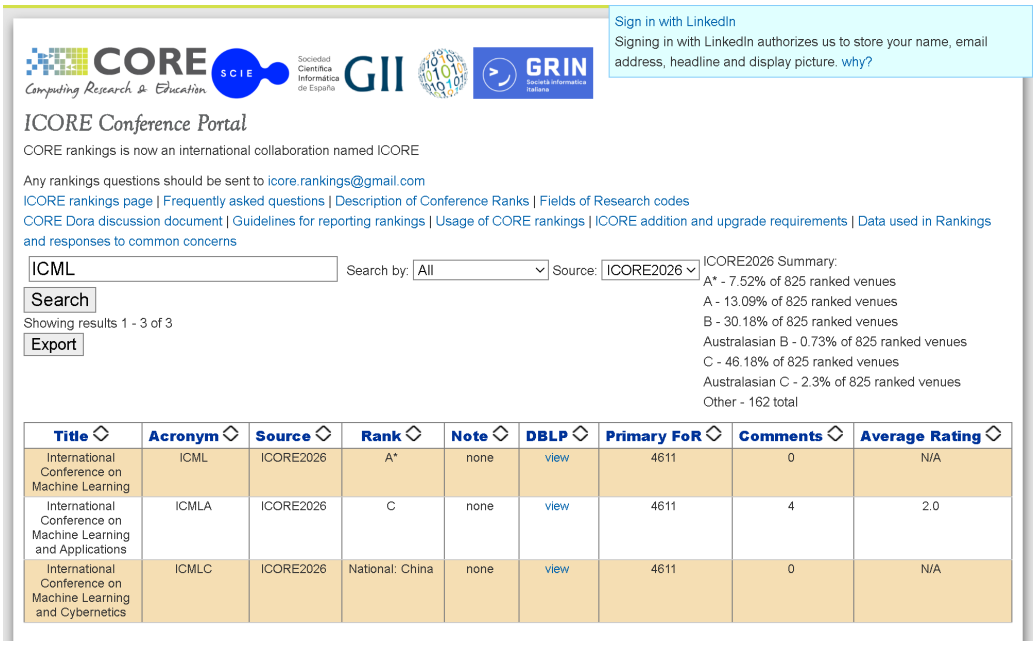
This review covers ten peer-reviewed papers from 2025 and 2026 on using large language models (LLMs) to turn natural-language problem descriptions into optimization models that a solver can run. Four papers appeared at the International Conference on Machine Learning (ICML) 2026 [1], [2], [3], [4], two at the Conference on Neural Information Processing Systems (NeurIPS) 2025 [5], [6], one at ICML 2025 [7], two at the International Conference on Learning Representations (ICLR) 2025 [8], [9], and one in the journal *Operations Research* in 2025 [10]. We found the papers through keyword searches of arXiv, OpenReview and conference proceedings, and confirmed each paper's venue and year on its official page. ICML, NeurIPS and ICLR are all ranked A* in the ICORE2026 conference ranking. Four further works are cited as supporting evidence but are not counted among the ten: a benchmark audit [11] and three recent papers close to our gaps [12], [13], [14], of which only [12] has been peer-reviewed so far. Sections 1.1 to 1.4 analyze the base paper, summarize the other nine papers, compare all ten and derive the three research gaps.

1.1 Base Paper Selection and Validation

1.1.1 Full Reference and Ranking Proof

Y. Chen, C. Cheng, Y. Sun, Z. Ling, and D. Ge, "OPT-Engine: Benchmarking the limits of LLMs in optimization modeling via complexity scaling," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, Seoul, South Korea, Jul. 2026, vol. 306, pp. 13807–13834 [1].

The paper is in the ICML 2026 proceedings (Proceedings of Machine Learning Research, volume 306). ICML is ranked **A*** in ICORE2026 (Figure 1), so the base paper meets the requirement of a 2025 or 2026 paper at an A*/A-ranked conference.



Title	Acronym	Source	Rank	Note	DBLP	Primary FoR	Comments	Average Rating
International Conference on Machine Learning	ICML	ICORE2026	A*	none	view	4611	0	N/A
International Conference on Machine Learning and Applications	ICMLA	ICORE2026	C	none	view	4611	4	2.0
International Conference on Machine Learning and Cybernetics	ICMLC	ICORE2026	National: China	none	view	4611	0	N/A

Figure 1: Ranking proof for the base paper venue. ICML is ranked A* in ICORE2026 (portal.core.edu.au, accessed 17 October 2026).

1.1.2 Justification for Selection

We selected OPT-Engine for four reasons.

1. **Same task.** An LLM reads a plain-language optimization question, writes a model, and a solver computes the answer. Its Solver-Integrated Reasoning (SIR) setting is the natural single-agent baseline for our system.
2. **Reliable, controlled data.** Generators compute exact reference optima, size parameters control difficulty in each of its ten problem classes, and the evaluation scripts are released. This reduces the labeling errors reported for several older benchmarks (Section 1.2).
3. **Fit with all three gaps.** Its linguistic perturbation set provides reworded questions for paraphrase checks (Gap 1), its ten classes allow leave-one-class-out tests of a refusal rule (Gap 2), and its comparison of reasoning paradigms without matched token budgets motivates an equal-budget study (Gap 3).
4. **Reproducible comparison.** We can run its SIR setting with an open model, so our results are comparable on the same data and metric.

1.1.3 Core Problem

Existing benchmarks for LLM-based optimization modeling are fixed collections of mostly textbook-style problems, so problem size and difficulty cannot be changed in a controlled way. It is therefore unclear at what complexity LLMs start to fail, and whether the cause is reasoning, arithmetic or constraint formulation. OPT-Engine asks three questions: does pure-text reasoning stay reliable as problems grow, can external tools fix its arithmetic weaknesses, and what limits the solver-integrated approach?

1.1.4 Agent Architecture

OPT-Engine is a benchmark paper and does not propose a multi-agent system. It compares three single-agent reasoning paradigms that differ in the tools given to the LLM:

- **Pure-Text Reasoning (PTR):** the LLM reasons step by step in text and states the final answer, with no code execution.
- **Computational PTR (CompPTR):** the LLM may run Python with computational libraries such as scipy, but has no global optimization solver.
- **Solver-Integrated Reasoning (SIR):** the LLM writes a model in code, and an exact solver (for example Gurobi) computes the optimum. This is a single agent with one tool.

Four models are tested: DeepSeek-V3.2, GPT-5.1, Qwen3-4B-Instruct and Qwen3-4B-RL (a 4B model trained further with reinforcement learning).

1.1.5 Datasets

The released data, named C^3-Bench (Controllable and Configurable Complexity Benchmark) in the authors' repository, come from parametric generators. A difficulty parameter, such as the number of cities in the traveling salesman problem (TSP), sets the complexity of each instance, and a solver computes its exact optimum. The ten problem classes are five linear programs (LP: Inventory, Portfolio, Production, Transportation and Pollution Control) and five mixed-integer programs (MIP: TSP, Knapsack, Bin Packing, Job-Shop Scheduling and Min-Cost Network Flow). Three perturbation sets test robustness on Inventory, Knapsack and TSP: *linguistic complexity* (the same instance in easy, moderate and hard wording), *objective perturbation* (a constant added to the objective, which leaves the optimal solution unchanged), and *constraint augmentation* (a few extra constraints with no new variables). The release contains 1,810 canonical problems (from 50 to 620 per class) and 3,760 perturbation rows. A prediction counts as correct when its relative error is below 10^{-3} (the released evaluation code uses 0.2%; see Section 2.10).

1.1.6 Key Results

1. **Solvers remain necessary.** With DeepSeek-V3.2, on TSP at the Hard level, accuracy is 2.0% with PTR, 44.0% with CompPTR and 80.0% with SIR. On Min-Cost Network Flow, PTR accuracy is 10.0% at Easy and 0.0% at Medium and Hard, while SIR reaches 100% at every level.
2. **Constraint formulation is the main weakness.** Linguistic and objective perturbations leave accuracy roughly stable, but constraint augmentation causes a consistent accuracy drop for both PTR and SIR. The authors conclude that computational tools help with local calculations but do not make the model respect global constraints.
3. **Small models struggle to produce runnable code.** Qwen3-4B-Instruct produces executable code for only 23.1% of TSP and Knapsack problems, against 86.4% and 100% for DeepSeek-V3.2. Reinforcement learning helps: Qwen3-4B-RL outperforms PTR across the complexity dimensions.

1.1.7 Limitations

Stated by the authors.

- Only exact-solution modeling is covered, although heuristics are often preferred for large instances.
- Tracks for optimality gaps, run time and heuristic design are future work.
- Only linear structures (LP and MIP) are included; nonlinear, stochastic and dynamic programs are left for future work.

Identified by our group.

- The paper measures failures but gives no way to detect them at run time: generated models are not verified, and there is no abstention, escalation or multi-agent design.
- Accuracy is the main quality measure; feasibility, optimality gap and execution rate appear only for some settings. The paper does not report how often a wrong answer would be accepted, and gives no confidence intervals.
- Token budgets are not matched across paradigms. The main results use one maximum response length of 16,128 tokens, but actual token use is not controlled.
- Only four models are tested, with a single small open model family.
- Perturbations cover only three classes, and the ten problem classes are textbook models in short scenario stories, not real multi-step operational workflows.
- Results are reported per class, but nothing estimates how reliable a system is on a class it has not seen before, which is the setting a deployed refusal rule must face.

1.1.8 How Our Project Builds On, Compares Against and Goes Beyond the Base Paper

Builds on. We use C^3-Bench and its perturbation sets as our main evaluation data, adopt its SIR setting as our baseline (with PuLP, a Python library for writing optimization models, and the open-source solver HiGHS in place of Gurobi), and grade answers with the tolerance in its released evaluation code.

Compares against. We run the SIR setting with an open model (gpt-oss-120b) and compare it with our system on the same problems and metric, class by class.

Goes beyond. (1) We add a verifier with label-free, requirement-level signals (Gap 1): *number coverage*, the share of the question's numbers that appear in the model code; *requirement coverage* (added in Phase 2), the share of parsed requirements that the code implements as named constraints; and *paraphrase-optimum agreement*, the share of independently solved rewordings that reach the same optimum. (2) We add a decision layer that answers, abstains or escalates, with a threshold calibrated to a target *wrong-accept rate* (the share of answered problems that are wrong) and tested on held-out problem classes (Gap 2). (3) We compare single-agent and multi-agent designs at equal token budgets (Gap 3). (4) We build OpsOR-Bench, a small set of operations problems generated from parameterized scenarios, as an extra test of distribution shift.

1.2 Critical Summaries of Nine Additional Papers

Each summary covers the problem, method, data, results and limitations (split into those stated by the authors and those identified by our group), and ends with its relevance to this project.

1.2.1 Opt-Verifier [2] (ICML 2026)

Problem. LLM-generated optimization models often miss implicit constraints, and running the code cannot reveal such formulation errors.

Method. A fixed multi-agent workflow distills the structure of the question to guide formulation, then checks the model from two sides: a structure-side evaluator compares an interpretation of the model with

this structure, and a solution-side evaluator judges an explanation of the solver output. Each gives a binary verdict, and a refinement agent revises the model. The main comparison uses GPT-4o-mini.

Data. NL4Opt (289 problems), MAMO ComplexLP (211), a 19-problem ComplexOR subset, IndustryOR (100) and OptMATH-Bench (166).

Results. Average solving accuracy is 64.3%, against 50.6% for OptiMUS with the same backbone, at about 9 agent calls and 5,300 to 8,500 tokens per problem. On hard IndustryOR problems, the structure-side check has 83% precision and 68% recall.

Limitations. *Stated by the authors:* none, as the paper has no limitations section. *Identified by our group:* the verifiers are tested on 30 problems, with 270 wrong models made by deleting or rewriting parts of correct ones; verdicts are binary and uncalibrated, so they cannot support refusal at a known error rate; its structure check ignores numerical values; cost is compared only with multi-agent systems.

Relevance. Its precision and recall give a reference point for our verifier signals (Gap 1), and its token figures inform the equal-budget study (Gap 3).

1.2.2 MIPLIB-NL: Constructing an Industrial-Scale Benchmark [3] (ICML 2026)

Problem. Benchmarks hold small or synthetic problems, while industrial mixed-integer programs are far larger, so benchmark accuracy says little about practice.

Method. Experts recover indexed variable groups and constraint families from MIPLIB 2017 solver files and describe them in text. Several LLMs try to rebuild each model from the text and data alone (up to eight attempts). Of 245 candidates, 163 passed and experts kept 60 more.

Data. MIPLIB-NL: 223 instances with text, data files, a model and code; 14 systems are scored by pass@1 (first sample correct) and pass@8 (one of eight samples correct).

Results. The best system, GPT-5.1, solves 39.05% (mean 17.85%). SIRL-32B scores 87.40% on MAMO-Complex but 1.43% here. Failures are mostly modeling (48.3%) and execution (45.0%) errors.

Limitations. *Stated by the authors:* they estimate that only about 40% of MIPLIB 2017 could be converted; validation is existential (one successful LLM reconstruction is enough); experts check the structure, not every detail; the 8 infeasible and 15 open instances are not scored. *Identified by our group:* it is a benchmark only, with no verifier, agentic baseline, token-cost measurement or refusal study.

Relevance. Fine-tuned models fail on these industrial problems, the shift a refusal rule must survive (Gap 2); omitted constraints motivate coverage checks (Gap 1).

1.2.3 VeriSimpl [4] (ICML 2026)

Problem. Generated optimization programs often run but do not implement the stated task, which execution alone cannot reveal.

Method. Up to ten sampled programs are checked with simplified solver queries. Each constraint is mutated three ways (satisfied with a margin, tight or violated), and the LLM judges from the text whether the solver's example solution for each version is feasible. The best-scoring program is returned.

Data. NL4Opt (269 problems), NLP4LP (62), ComplexOR (17) and IndustryOR (96), with GPT-4o, DeepSeek-R1 and Mistral Large.

Results. With GPT-4o, average accuracy rises from 56.6% to 65.5% and with DeepSeek-R1 from 62.5% to 72.8%. Fully verified GPT-4o programs are 91.5% correct, but only 34.2% of correct programs pass full verification (the paper calls this recall coverage); for DeepSeek-R1, 78.5% precision at 23.0% recall.

Limitations. *Stated by the authors:* the queries cannot detect requirements that a program omits entirely, verification passes when the program and the LLM share a misreading, and recall is low. *Identified by our group:* three test sets are small (17 to 96 problems), without confidence intervals; token cost is not reported; the precision-recall trade-off is not turned into a refusal rule.

Relevance. Its blind spot for omitted requirements is what our coverage signals target (Gap 1); high precision only at low recall motivates a calibrated threshold (Gap 2).

1.2.4 OptiTree [5] (NeurIPS 2025)

Problem. Fixed-step decomposition fails on complex operations research problems.

Method. A modeling tree organizes problem types from general to specific, and each node stores statement and modeling "thoughts." At test time, LLM-guided tree search finds related subproblems, whose thoughts are merged into guidance for the formulation. The tree is built from 400 OR-Instruct problems and updated when a formulation does not match the ground truth.

Data. Seven benchmarks (NL4Opt, MAMO Easy and Complex, ComplexOR, IndustryOR, OptiBench and OptMATH-Bench), with GPT-4o and DeepSeek-V3.

Results. With DeepSeek-V3, OptiTree reaches 81.5% on MAMO-Complex, 54.0% on IndustryOR and 52.4% on OptMATH-Bench, against 51.6%, 46.0% and 38.6% for the strongest prompt-based baseline with the same model, Monte Carlo tree search (MCTS).

Limitations. *Stated by the authors:* most remaining errors are missing constraints (from 61.9% on OptMATH-Bench to 82.6% on IndustryOR), and thought synthesis depends on the reasoning of the base model. *Identified by our group:* tree updates rely on ground-truth labels; possible overlap between OR-Instruct and the test sets is not checked; token use and results on unseen problem classes are not reported.

Relevance. Its residual errors are strong evidence for requirement-level checks (Gap 1), and its missing token figures show why budgets must be measured (Gap 3).

1.2.5 SIRL: Solver-Informed Reinforcement Learning [6] (NeurIPS 2025)

Problem. LLMs often produce flawed or infeasible optimization models, which limits their use in practice.

Method. Reinforcement learning with verifiable rewards: the solver checks each generated model, and the reward combines format, execution and optimum accuracy, with a second stage rewarding advanced formulations. A partial Kullback-Leibler (KL) penalty applies only to the formulation and code. Qwen2.5-7B and 32B-Instruct are trained without a supervised stage on 10,000 synthetic questions. The training labels come from instance-enhanced self-consistency: candidate models vote using the objective value, optimization direction and variable counts from the solver's model file.

Data. NL4Opt, MAMO Easy and Complex, IndustryOR and OptMATH-Bench, scored by pass@1.

Results. SIRL-32B reaches 68.3% macro accuracy, above DeepSeek-V3.1 (65.0%) and o3 (57.1%).

Limitations. *Stated by the authors:* reward hacking still occurs, and IndustryOR and OptMATH-Bench remain hard. *Identified by our group:* the model always answers; in the evaluation of [3], SIRL-32B scores 87.40% on MAMO-Complex but only 1.43% on MIPLIB-NL, so its accuracy does not transfer to industrial problems; token cost is not reported.

Relevance. This sharp drop under shift motivates calibration on held-out classes (Gap 2), and its agreement-based labels support agreement as a correctness signal, which we use at inference (Gap 1).

1.2.6 OptMATH [7] (ICML 2025)

Problem. Good training data for optimization modeling are scarce, which limits what fine-tuned models can learn.

Method. Bidirectional synthesis. Instances come from about 53 seed generators and from solver files of benchmarks such as MIPLIB 2017; DeepSeek-V3 back-translates each instance into a question; an AutoFormulator writes a model for the question, and rejection sampling keeps the pair only if the optimum

matches. LLM augmentation adds about 50,000 samples, and Qwen2.5 models from 0.5B to 32B are fine-tuned with low-rank adaptation (LoRA).

Data. OptMATH-Train (over 150,000 samples, 53 problem types) for training, and the new OptMATH-Bench together with NL4Opt and MAMO for testing, scored by pass@1.

Results. OptMATH-32B reaches 68.7% macro accuracy, slightly above DeepSeek-V3 at 67.0%. A 1% manual check finds 99.6% of kept instances correct.

Limitations. *Stated by the authors:* a matching optimum does not prove that two formulations are equivalent. *Identified by our group:* optimum matching is used only to filter training data, so nothing checks models at inference time; token cost is not reported; OptMATH-7B scores 0.00% on MIPLIB-NL [3].

Relevance. Our paraphrase-optimum agreement applies the same optimum-matching idea at inference time and without labels (Gap 1), and the failure on industrial problems supports testing under shift (Gap 2).

1.2.7 LLMOPT [8] (ICLR 2025)

Problem. Current LLM approaches to optimization modeling lose accuracy and cover only a narrow range of problem types.

Method. A five-element formulation (sets, parameters, variables, objective and constraints) gives the task a fixed structure. Qwen1.5-14B is fine-tuned on 1,763 expert-reviewed seed problems expanded by seven instruction types, then aligned with Kahneman-Tversky Optimization (KTO). At inference, an auto-testing loop runs the code, and the same model reviews the problem, formulation, code and output, then decides what to redo, up to 12 times.

Data. Six benchmarks (NL4Opt, MAMO Easy and Complex, IndustryOR, NLP4LP and ComplexOR), with 100-problem test sets for the large ones; metrics include execution rate and solving accuracy.

Results. Solving accuracy improves by 11.08 points on average over earlier methods, reaching 68.0% on MAMO-Complex and 46.0% on IndustryOR. Without self-correction, MAMO-Complex accuracy falls from 68.0% to 32.0%.

Limitations. *Stated by the authors:* expert labeling is costly, some IndustryOR labels are wrong, and problems whose data sit in databases or files are not yet handled. *Identified by our group:* the same model checks its own work, so an omission it does not notice passes unchecked; 100-problem test sets give wide confidence intervals; there is no refusal and no token-cost report.

Relevance. The large drop without self-correction shows how much results depend on extra calls (Gap 3), and self-review by the same model motivates an independent requirement check (Gap 1).

1.2.8 OptiBench and ReSocratic [9] (ICLR 2025)

Problem. Existing benchmarks are mostly small and linear and rarely include tables, and varied training data cannot be produced at scale.

Method. ReSocratic synthesizes data in reverse. From 27 seed examples, DeepSeek-V2 writes new formal models step by step; filters remove malformed and near-duplicate ones. Code is generated and run, failures are deleted, and each remaining model is turned back into a question, giving ReSocratic-29k for fine-tuning.

Data. OptiBench: 605 manually checked questions of four types (linear or nonlinear, with or without tables); an answer counts only if all variables and the objective match.

Results. Fine-tuning on ReSocratic-29k raises Llama-3-8B-Instruct from 13.6% (few-shot) to 51.1%. Zero-shot GPT-4 reaches 62.8%. Averaged over GPT-4 and GPT-3.5-Turbo, accuracy is 66.9% on linear but only 30.8% on nonlinear problems.

Limitations. *Stated by the authors:* the paper has no limitations section; it focuses mainly on explicit problems, whose numbers appear in the question, and its results show that table, nonlinear and longer

questions remain harder. *Identified by our group:* data are checked by hand only in small samples; nothing is verified or refused at inference; token cost is not reported; testing is mainly on its own benchmark.

Relevance. Questions with tables test number coverage, since every value in a table should appear in the model (Gap 1).

1.2.9 ORLM [10] (*Operations Research*, 2025)

Problem. Optimization modeling needs scarce expertise, and closed models such as GPT-4 are costly and raise privacy concerns.

Method. OR-Instruct expands 686 industrial seed cases over two rounds, adding scenarios and question types and changing objectives, constraints and wording. Filters drop about 39% of new examples per round, leaving 32,481. Open 7B and 8B models, including LLaMA-3, are fine-tuned on it to write programs for the COPT solver.

Data. NL4Opt, MAMO Easy and Complex, and the new 100-problem IndustryOR, scored by execution accuracy (greedy pass@1; micro and macro averages).

Results. ORLM-LLaMA-3-8B reaches 71.4% micro and 60.8% macro accuracy, against 50.2% and 39.1% for GPT-4 prompting; with eight samples (pass@8), micro accuracy reaches 83.6%. In 100 manually analyzed failures, 56.3% of modeling errors come from low completeness: ignored constraints or implicit conditions.

Limitations. *Stated by the authors:* the models do poorly on nonlinear and rare problem types, and a correct answer often appears among eight samples but not first. *Identified by our group:* the synthetic data are only 70% to 75% accurate; there is no verification, refusal or token-cost report; an audit finds 50% of IndustryOR and 41% of OptMATH-Bench instances incorrect [11]; ORLM-LLaMA-3-8B scores 0.0% on OptMATH-Bench [7].

Relevance. The gap between pass@1 and pass@8 is headroom that selective answering can capture (Gap 2), and the dominant completeness errors motivate coverage checks (Gap 1).

1.3 Cross-Paper Comparison

The ten papers differ most along three points that matter for this project. **Where checking happens:** four papers check only while training or building data: SIRL [6], OptMATH [7], OptiBench [9] and ORLM [10]. At inference, LLMOPT [8] lets the same model review its own work. Only Opt-Verifier [2] and VeriSimpl [4] check the meaning of the generated model separately, using uncalibrated LLM verdicts [2] or solver queries that cannot see omitted requirements [4]. **What happens after checking:** all ten systems answer every problem, and none can abstain or escalate with a known error rate. **Cost:** only Opt-Verifier [2] reports tokens per problem, OPT-Engine [1] reports one token trend, and no study compares designs at the same budget. Our project covers all three points: it checks every model at inference, abstains or escalates under a tested error rate, and meters the tokens of every call. Table 1 compares all ten papers, with the base paper in row 1.

1.4 Research Gap Synthesis

From the limitations in Sections 1.1 and 1.2, we identified five candidate gaps:

(a) none of the ten reviewed papers checks, at inference time and without labels, whether the generated model uses every number in the question or whether reworded questions reach the same optimum;

(b) none of the ten reviewed systems can refuse to answer with a controlled error rate, and none tests such a rule on unseen problem types;

(c) none of the ten reviewed studies compares single-agent and multi-agent designs at the same token budget;

Table 1: Cross-paper comparison of the ten reviewed papers. Row 1 is the base paper.

#	Title	Year & Venue	Dataset / Benchmark	Method / Agent Architecture	Evaluation Metrics	Key Results	Limitations / Research Gap
1	**Base paper.** OPT-Engine: Benchmarking the limits of LLMs in optimization modeling via complexity scaling [1]	2026, ICML (A*)	C^3-Bench: 10 generated LP/MIP classes with size-controlled complexity; perturbation sets for 3 classes; 1,810 canonical problems released	Benchmark comparing three single-agent paradigms: PTR (text only), CompPTR (text and Python tools), SIR (LLM writes solver code)	Accuracy (relative error below 10^{-3}); feasibility, optimality gap and execution rate in some settings	TSP Hard with DeepSeek-V3.2: SIR 80.0%, CompPTR 44.0%, PTR 2.0%; added constraints lower accuracy	No verifier, refusal or multi-agent design; no wrong-accept rate; token budgets not matched
2	Opt-Verifier: Unleashing the power of LLMs for optimization modeling via dual-side verification [2]	2026, ICML (A*)	NL4Opt, MAMO ComplexLP, ComplexOR (19), IndustryOR, OptMATH-Bench	Multi-agent: structure-side and solution-side LLM evaluators with binary verdicts, plus a refinement agent (GPT-4o-mini in the main comparison)	Solving accuracy; verifier precision and recall; tokens and calls	Average 64.3% against 50.6% for OptiMUS; about 9 calls and 5,300 to 8,500 tokens per problem	Uncalibrated binary verdicts; verifiers tested on 30 problems; no refusal; no single-agent control at equal budget
3	Constructing industrial-scale optimization modeling benchmark (MIPLIB-NL) [3]	2026, ICML (A*)	MIPLIB-NL: 223 instances reverse-built from MIPLIB 2017	Benchmark built by expert-guided, structure-aware reverse construction; 14 systems evaluated	pass@1, pass@8	Best 39.05% (GPT-5.1), mean 17.85%; SIRL-32B 87.40% on MAMO-Complex but 1.43% here	Benchmark only, no verifier; existential validation; about 40% of MIPLIB 2017 convertible (authors' estimate); sharp drops under distribution shift; no token-cost measurement
4	VeriSimpl: Robust optimization modeling from natural language using simplification-based verification [4]	2026, ICML (A*)	NL4Opt (269), NLP4LP (62), ComplexOR (17), IndustryOR (96)	Solver-generated diagnostic queries judged by the LLM; best of up to 10 candidate programs	Accuracy; precision and recall of verified programs (recall is called coverage)	GPT-4o: 65.5% against 56.6% for the base model; 91.5% precision at 34.2% recall	Cannot see omitted requirements; low recall; no token-cost report or refusal rule
5	OptiTree: Hierarchical thoughts generation with tree search for LLM optimization modeling [5]	2025, NeurIPS (A*)	Seven benchmarks, including MAMO, IndustryOR, OptiBench and OptMATH-Bench	Modeling tree and tree search that merge the modeling thoughts of related subproblems (prompting)	Solving accuracy	DeepSeek-V3: MAMO-Complex 81.5% against 51.6% for the strongest same-model baseline (MCTS); IndustryOR 54.0% against 46.0%	61.9% to 82.6% of residual errors are missing constraints; no refusal; no token figures
6	Solver-informed RL: Grounding large language models for authentic optimization modeling [6]	2025, NeurIPS (A*)	NL4Opt, MAMO, IndustryOR, OptMATH-Bench	Reinforcement learning with solver-verified rewards and partial KL; training data labeled by instance-enhanced self-consistency	pass@1, macro average	SIRL-32B: 68.3% macro, above DeepSeek-V3.1 (65.0%) and o3 (57.1%)	Always answers; no inference-time check; 1.43% on MIPLIB-NL [3]; no token-cost report
7	OptMATH: A scalable bidirectional data synthesis framework for optimization modeling [7]	2025, ICML (A*)	OptMATH-Train (over 150,000 samples); OptMATH-Bench, NL4Opt, MAMO	Bidirectional data synthesis with optimum-match rejection sampling; LoRA fine-tuning	pass@1, macro average	OptMATH-32B: 68.7% macro against 67.0% for DeepSeek-V3	Optimum match is not equivalence; no inference-time check; no token-cost report; 7B model scores 0.00% on MIPLIB-NL

Continued on the next page

Table 1 (continued)

#	Title	Year & Venue	Dataset / Benchmark	Method / Agent Architecture	Evaluation Metrics	Key Results	Limitations / Research Gap
8	LLMOPT: Learning to define and solve general optimization problems from scratch [8]	2025, ICLR (A*)	Six benchmarks, 100-problem test sets for the large ones	Five-element formulation; multi-instruction fine-tuning and KTO; self-correction by the same model, up to 12 times	Execution rate, solving accuracy, average solving rounds	+11.08 points on average; MAMO-Complex 68.0% (32.0% without self-correction)	Self-review by the same model; small test sets; no refusal; no token-cost report
9	OptiBench meets ReSocratic: Measure and improve LLMs for optimization modeling [9]	2025, ICLR (A*)	OptiBench (605 linear and nonlinear problems, with and without tables); ReSocratic-29k	Reverse data synthesis and supervised fine-tuning	Exact match of all variables and the objective	Llama-3-8B: 13.6% to 51.1% after fine-tuning; GPT-4 zero-shot 62.8%	No inference-time verification or refusal; data checked by hand only in small samples; no token-cost report
10	ORLM: A customizable framework in training large models for automated optimization modeling [10]	2025, *Operations Research* (journal)	OR-Instruct (32,481 examples); NL4Opt, MAMO, IndustryOR (introduced here)	Semi-automated data synthesis; fine-tuning of 7B and 8B open models	Execution accuracy (micro and macro); pass@8	LLaMA-3-8B: 71.4% micro, 60.8% macro; pass@8 micro 83.6%	Large gap between pass@1 and pass@8; 56.3% of modeling errors are completeness errors; no refusal; no token-cost report

(d) several public benchmarks have unreliable reference answers [11];

(e) benchmarks lack realistic operational data and multi-step workflows; MIPLIB-NL [3] adds industrial scale but not operational workflows.

We select (a), (b) and (c) as the three strongest gaps; they become Gap 1, Gap 2 and Gap 3. Each is **specific** (it names one missing capability), **evidence-based** (it rests on named results in Table 2), **relevant** (it extends the SIR setting of the base paper on its own benchmark) and **testable** (Section 2.12 defines metrics we can compute this semester on C^3-Bench, calling the model through an application programming interface (API)). Candidate gap (d) is a data-quality problem that we reduce by using generated problems with computed optima; the Phase 2 failure analysis also checks doubtful reference answers by hand. Candidate gap (e) is handled by our development contribution, OpsOR-Bench, which also gives Gap 2 an extra shift test.

Table 2: Research gap table: the three selected gaps.

Gap	Supporting evidence (papers)	Proposed approach	How it will be measured
Gap 1. Requirement-level verification	OptiTree [5]: 61.9% to 82.6% of residual errors are missing constraints. ORLM [10]: 56.3% of modeling errors come from ignored constraints or implicit conditions. MIPLIB-NL [3]: 48.3% of failures are modeling errors, such as omitted constraints or wrong links between variables and data. VeriSimpl [4]: its queries cannot see omitted requirements, and only 23.0% to 34.2% of correct programs pass verification. Opt-Verifier [2]: binary LLM verdicts with 68% recall on hard structure errors; its structure check ignores numerical values. OPT-Engine [1]: added constraints lower accuracy.	Three label-free signals in the verifier: *number coverage* (share of the numbers in the question that appear in the model code), *requirement coverage* (Phase 2: share of the parser's requirements that the formalizer links to a named constraint) and *paraphrase-optimum agreement* (share of k independently solved rewordings that reach the main model's optimum). With solver feasibility, they form one score: the mean of the available signals, or 0 when no feasible model exists.	AUROC (area under the receiver operating characteristic curve) of the score for separating correct from wrong answers, compared with feasibility alone (target: at least 0.10 higher); precision and recall of accepted answers; selective accuracy against the base-paper SIR setting; ablation of each signal; on canonical and constraint-augmented problems, with the paraphrase signal also checked on 30 linguistic-split problems.
Gap 2. Risk-controlled abstention under distribution shift	None of the ten systems can abstain. SIRL-32B [6] falls from 87.40% on MAMO-Complex to 1.43% on MIPLIB-NL in the evaluation of [3]. VeriSimpl [4] reaches 78.5% to 91.5% precision only at 23.0% to 34.2% recall, with no threshold rule. OPT-Engine [1] has ten classes but no held-out-class study. AdmitOR [14] (preprint): a finite-sample bound held on calibration problems but failed on a benchmark stream (15.9% wrong against a 5% target, mostly from questions with missing or rounded numbers).	A decision layer that answers, abstains or escalates with reasons. Thresholds that accept at least 29 calibration problems (fewer cannot give a bound below 10%) are tested from strict to lenient. The scan stops before the first threshold whose one-sided 95% Clopper-Pearson (exact binomial) upper bound on the calibration wrong-accept rate exceeds the target wrong-accept rate $\alpha = 10\%$; this fixed order keeps the bound valid across thresholds. If no threshold passes, the fold answers nothing. Leave-one-class-out: calibrate on nine classes and test on the tenth.	Pooled held-out wrong-accept rate (each problem is tested once, in the fold that holds out its class) with an exact 95% interval, compared with α; coverage (share answered, target at least 50%); risk-coverage curves; comparison with a fixed threshold and with the base-paper SIR setting, which never abstains; OpsOR-Bench as a further shift test.

Gap	Supporting evidence (papers)	Proposed approach	How it will be measured
Gap 3. Equal-budget single-agent versus multi-agent comparison	Opt-Verifier [2] reports about 9 calls and 5,300 to 8,500 tokens per problem but compares only with other multi-agent systems. LLMOPT [8]: MAMO-Complex accuracy falls from 68.0% to 32.0% without self-correction, so extra calls change results greatly, yet no tokens are reported. ORLM [10]: pass@8 (83.6%) far exceeds pass@1 (71.4%), so more samples alone help a single model. MIPLIB-NL [3], OptiTree [5], SIRL [6], OptMATH [7], VeriSimpl [4] and OptiBench [9] report no token cost. OPT-Engine [1] compares paradigms without matching token budgets.	Meter input tokens, output tokens and calls on every LLM call. Build a single-agent variant, B2, with the same model and decision rule: it formalizes, repairs and resamples until it uses the pipeline's mean token budget, answers with the most frequent optimum, and uses the agreement rate as its score.	Accuracy and wrong-accept rate at equal coverage (50%) from risk-coverage curves, at matched mean tokens per problem (within 10%) for three budget levels ($k = 1, 2, 3$), with the main test at $k = 3$; paired McNemar tests and bootstrap intervals on the same problems; accuracy-cost curves; latency.

Closest recent work. Three recent papers outside the ten reviewed ones touch the same gaps. ReLoop [12] (accepted at NeurIPS 2026) and a falsification-based test battery [13] (preprint) check a generated model by changing its input data and testing whether the solver's answer responds as the text implies. Neither turns its checks into a refusal rule with an error target. AdmitOR [14] (preprint) answers, abstains or escalates using agreement across three model families, with a finite-sample bound on wrong acceptances. The bound held on its calibration problems but failed on a benchmark stream, where 22 of 138 accepted answers (15.9%) were wrong, above both its 5% target and its 10% budget. Its authors trace 20 of these 22 cases to question texts that omit needed numbers (15 cases) or print them with too little precision (5 cases). Their label-free check for values missing from the text flags 9 of the 15 omitted-number cases but none of the 5 rounded ones. This supports Gap 2, since error control fitted on one distribution must be tested on a shifted stream, and Gap 1, since data-level checks can catch some of these errors. AdmitOR uses no paraphrases of the question and reports no token cost.

2 Project Proposal

2.1 Project Title and Selected Seed

Selected seed. LLM Agents that Translate Natural Language into Verified Optimization Models (Project G1-01, supervised by Muhammad Sajjad).

Our title. Verified Optimization Agents: Requirement-Level Verification and Calibrated Abstention for LLM-Based Optimization Modeling.

Scope. The seed asks for four parts: a benchmark of realistic operations problems, requirement-level verification, a calibrated refusal layer, and a comparison of single-agent and multi-agent designs at equal compute, which we measure as an equal token budget. We keep all four. Verification, refusal and the equal-budget comparison are our three research gaps (Gaps 1, 2 and 3). The benchmark, OpsOR-Bench, is a development contribution that also serves as an out-of-distribution test. The main experiments run on the base paper's C^3-Bench, so results stay comparable with published work. A verified answer here means one that passed the system's checks, not a formal proof of correctness.

2.2 Problem Statement and Motivation

Problem. Operations teams describe planning problems in plain language, for example: "Can we handle the 17:00 arrival wave if two de-icing trucks are out of service?" (an example from the project seed). Given the flight schedule and fleet data, answering such a question needs a formal optimization model. LLMs can now write these models as solver code, and fine-tuned systems reach about 60% to 70% macro accuracy on standard benchmarks [6], [7], [10]. Accuracy drops sharply on harder and more realistic

problems: the best model solves only 39% of the industrial MIPLIB-NL instances [3], and SIRL-32B [6] falls to 1.43% there [3].

Current limitations. Current systems always answer. A generated model that runs and returns an "optimal" value can still miss a stated requirement: missing constraints are the most common error type in [5] and [10] and a common modeling error in [3]. Existing checks are weak: self-review by the same model [8], uncalibrated LLM judges [2], or solver queries that miss omitted requirements [4]. None of the ten reviewed systems can say "I am not confident; please check this case" with a known error rate, and the closest recent attempt saw its error bound fail on new problems [14].

Target users. The target users are operations analysts and planners in airport ground operations, logistics, production and supply-chain planning. They understand their problems but are not optimization experts. A second group is operations research engineers who want a reliable first draft of a model.

Why it matters. In operations, a confident wrong plan is worse than no plan. A silently dropped constraint, such as a crew limit or an unavailable vehicle, can produce a schedule that cannot be carried out, with real cost and safety consequences. Our system therefore answers only when its checks support the answer, and otherwise abstains or hands the case to a human analyst with clear reasons.

2.3 Base Paper and Research Gaps Summary

OPT-Engine [1] (ICML 2026, A*) builds a controlled benchmark of ten problem classes and shows that, with large models such as DeepSeek-V3.2, solver-integrated reasoning (SIR), in which the LLM writes model code for a solver, stays far more reliable than text-only reasoning as complexity grows. It also identifies constraint formulation as the main remaining weakness. However, it does not detect wrong models, cannot refuse and does not control token budgets. Our project addresses the three gaps in Table 2:

- **Gap 1:** add requirement-level verification signals to the SIR setting;
- **Gap 2:** add calibrated abstention and test it on held-out problem classes;
- **Gap 3:** compare single-agent and multi-agent designs at equal token budgets.

Beyond the base paper, we also add OpsOR-Bench, a set of operations problems generated from parameterized scenarios, to test behavior outside C^3-Bench.

2.4 Target Product or System

By the end of the semester, the group will deliver **Verified Optimization Agents**, an open-source Python system with five parts:

1. **A usable application:** a command-line application and a simple interactive interface for the live demonstration. A user enters an optimization question in plain language. The system returns either a checked answer (optimal value, solution plan, model code and a short explanation) or an abstention or escalation to a human analyst with specific reasons, such as numbers the model does not use or rewordings that lead to different optima.
2. **A configurable decision layer:** the operator sets the target wrong-accept rate.
3. **A reproducible evaluation harness:** fixed data samples, one command per experiment, saved run records, summary tables and plots.
4. **OpsOR-Bench:** at least 30 operations problems in three families (gate assignment, ground-crew rostering and equipment dispatch), generated from parameterized scenarios with all data stated in the question, together with reference models, reference optima and automatic grading.
5. **Documentation and tests:** setup and run instructions, and tests for all deterministic components.

2.5 Agentic Justification

Why not a deterministic workflow? The input is free text about arbitrary problems; no fixed program can turn such text into a correct model.

Why not a single LLM call? One call cannot run the solver, read the result, repair errors or check its own work. The base paper shows that, with large models, text-only reasoning degrades sharply as complexity grows, while solver-integrated reasoning stays far more accurate [1]. LLMOPT loses more than half of its MAMO-Complex accuracy (68.0% to 32.0%) without its self-correction loop [8].

Why not a retrieval-augmented generation (RAG) pipeline? The most common failure is a missing constraint or requirement in the model that the LLM wrote [5], [10]. Retrieval adds information but does not check whether the model covers every requirement.

What makes the system agentic. It pursues a goal, a trustworthy answer, through a loop whose path depends on what it observes. It calls an external tool (the solver), reads statuses, errors and objective values, and chooses the next step from them: a failed solve triggers a repair, an optimal solve triggers the paraphrase runs, and the verifier score decides whether to answer, abstain or escalate. Splitting the work into roles keeps the checks independent of the formalizer's own reasoning. We do not assume that more agents are better: Gap 3 tests this at an equal token budget.

2.6 Goals and Success Criteria

Table 3 lists the development objectives and the research objectives, with at least one measurable success criterion for each gap.

Table 3: Development and research objectives with success criteria.

ID	Gap	Objective	Success criterion
Development objectives			
D1	All three	Five-role pipeline with the decision layer, running end to end	Runs on every evaluation problem with fewer than 5% failed runs (crashes or rejected requests)
D2	Gap 2	Calibrated decision layer	Threshold computed automatically from calibration runs for an operator-set target α; per-class reports generated
D3	Gap 3	Equal-budget experiment harness	Token and call metering on every LLM call; single-agent variant matched to within 10% of the pipeline's mean tokens
D4	Gap 2	OpsOR-Bench	At least 30 problems in three families, with reference models and automatic grading
D5	All three	Usable application and reproducible repository	Command-line and interactive interface; one command per experiment; all tests pass; setup and configuration documented
Research objectives (one per gap)			
R1	**Gap 1**	Detect wrong models without labels	The combined score improves the AUROC over feasibility alone by at least 0.10, and removing any one signal lowers the AUROC
R2	**Gap 2**	Refuse with a controlled error rate under shift	With $\alpha = 10\%$, the pooled held-out wrong-accept rate over the ten leave-one-class-out folds has a point estimate of at most α (reported with its exact 95% interval), while at least 50% of all problems are still answered
R3	**Gap 3**	Fair single-agent versus multi-agent comparison	At the largest matched budget ($k = 3$), the pipeline's wrong-accept rate at equal coverage (50%) is lower than B2's, with the whole 95% bootstrap interval of the paired difference below zero

2.7 Data / Environment / Source

Primary data (confirmed and already in use). OPT-Engine C^3-Bench, downloaded by `scripts/download_data.py` from the authors' GitHub repository at a pinned commit:

- *Canonical split:* 1,810 problems in ten classes (Bin Packing 620; Knapsack 260; Inventory, Job-Shop Scheduling, Portfolio and TSP 160 each; Pollution Control, Production and Transportation 80 each; Min-Cost Network Flow 50).
- *Perturbation splits* for Inventory, Knapsack and TSP: linguistic complexity (1,640 rows in easy, moderate and hard wording), objective perturbation (1,060 rows), and constraint augmentation (1,060 rows); the last two each contain 530 original and 530 modified instances.

Every problem has a reference optimum, which is used only for grading and is never shown to any agent.

Evaluation samples. A fixed, stratified sample of 10 canonical problems per class (100 problems, random seed 0); 30 original and augmented pairs from the constraint-augmentation split (10 per class); and 30 problems from the linguistic split (10 per class), used to test whether our generated rewordings behave like the human-designed ones.

Secondary data (built in Phase 2). OpsOR-Bench, generated by Python problem generators with PuLP reference models, so that every reference optimum is computed exactly. The scenario parameters (fleet sizes, service times and flight waves) are set by the group within plausible ranges, and every question states all of its data.

Environment and tools. Python 3.12; PuLP 3 with the open-source HiGHS solver.

Models and APIs. `gpt-oss-120b`, an open-weight model, through Amazon Bedrock (model ID `openai.gpt-oss-120b-1:0`) is the main model for every role and every baseline. We first used the Groq free plan, but one member could not create a Groq API key, and its limits (200,000 tokens per day and 8,000 tokens per minute) are too small for the planned runs. Bedrock serves the same model weights without a daily limit at our scale, and each member uses the credits that Amazon Web Services (AWS) gives a new account. Phase 1 used temperature 0 for every call. In Phase 2, the paraphraser and the repeated samples of B2 use temperature 0.7, because repeated calls at temperature 0 can return almost the same text; all other calls stay at temperature 0. The client also works with any OpenAI-compatible endpoint, so another provider can replace Bedrock.

2.8 System Architecture Description

Models. One backbone LLM (`gpt-oss-120b`) serves every LLM role in both the baselines and the pipeline, so differences in results come from the architecture and not from the model.

Roles. Five roles plus a decision layer (Figure 2):

1. **Parser** (LLM): turns the question into a JavaScript Object Notation (JSON) specification with the objective sense, the decision variables (name, meaning and type), every numeric parameter with its unit, and a list of requirements, one per constraint.
2. **Formalizer** (LLM): writes PuLP code from the question and the specification, and is told that every listed requirement must appear as a constraint.
3. **Solver-runner** (tool): runs the code with HiGHS in a separate process with a time limit and a minimal environment (no API keys), and returns the status, objective value and any error text. If the status is not optimal, the error text goes back to the formalizer, at most r times (r repair rounds).
4. **Verifier**: checks feasibility and computes number coverage and paraphrase-optimum agreement; Phase 2 adds requirement coverage. For agreement, a paraphraser (LLM) rewrites the question k times, only if the main model is optimal, and each version is solved once with the baseline prompt. The score is the mean of the available signals, and 0 when no feasible model exists.

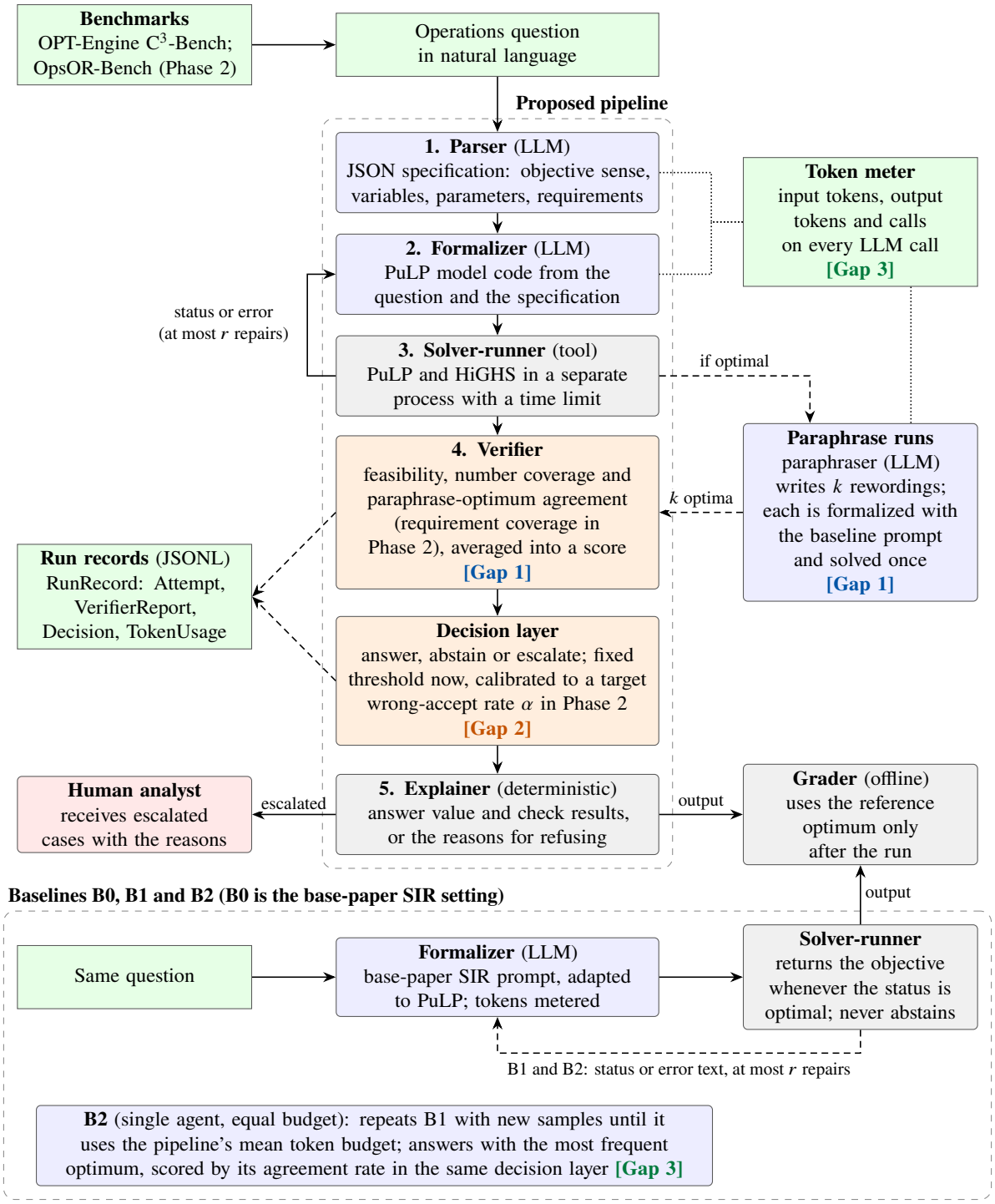


Figure 2: Architecture and methodology blueprint. Blue boxes are LLM roles, gray boxes are deterministic tools, orange boxes are the checking and decision components, green boxes are data and state, and the red box is the human analyst. Colored tags mark where each research gap is addressed. Solid arrows show the main flow, dashed arrows show conditional or auxiliary flows, and dotted lines show token metering. In the lower frame, B1 adds the repair loop to B0, and B2 (Section 2.10) adds resampling at an equal token budget.

5. **Explainer** (deterministic): writes the final message: the optimal value and check results, or the reasons for abstaining or escalating. Phase 2 adds the values of the decision variables.

The **decision layer** (deterministic) abstains when no feasible model is found after repairs, escalates when the score is below the threshold and answers otherwise. Phase 1 uses a fixed threshold of 0.8; Phase 2 calibrates it to a target wrong-accept rate α (Gap 2).

Tools. The solver-runner is the main tool: PuLP builds the model, and HiGHS solves it. The number-coverage check and the grader are deterministic Python functions.

State and contracts. All stages exchange typed records (Python dataclasses): `Problem`, `Attempt`, `SolveResult`, `VerifierReport`, `Decision`, `TokenUsage` and `RunRecord`. Each run is saved as one JSON line, so every decision can be traced and every experiment re-analyzed without new LLM calls.

Context. Each LLM call sees only what its role needs: the question, plus the specification for the formalizer, or the previous code and solver message for a repair; each paraphrase run sees only its rewording. No role sees the reference answer.

Knowledge and memory. No retrieval is needed: every question contains all its data. There is no memory across problems, which prevents leakage between test items; calibration uses saved run records.

Orchestration. A Python controller runs the roles in a fixed order with bounded loops (r repairs and k paraphrases), so cost is predictable and runs are reproducible. Within this order, repairs run only after a failed solve, paraphrase runs only after an optimal solve, and the final action depends on the verifier score.

Human controls. The operator sets the threshold (Phase 1) or the target wrong-accept rate α (Phase 2); escalated cases go to a human analyst with reasons; full traces support audit.

External environment. LLM provider APIs, the solver process and the file system for run records.

2.9 Methodology and Architecture Blueprint

Figure 2 shows the complete system, with the baselines in the lower frame and a colored tag where each gap is addressed. A question comes from C^3-Bench or OpsOR-Bench, and its reference optimum is kept aside for grading. The roles of Section 2.8 then run in order: parse and formalize, solve and repair, verify **[Gap 1]**, decide **[Gap 2]** and explain; escalated cases go to the human analyst with the explanation. Every LLM call adds its input tokens, output tokens and one call to the problem's totals **[Gap 3]**, and each run stores its latency, so the equal-budget experiments can use these totals. After the run, the grader compares the answer with the reference optimum using the tolerance of the base paper's released evaluation code. Compared with the baselines (Section 2.10), the pipeline adds the parser, the verifier checks, the calibrated decision layer and the explainer.

2.10 Baseline Definition

B0, base-paper setting (SIR). One LLM call writes PuLP code from the question using the base paper's zero-shot SIR prompt, adapted to PuLP and HiGHS in place of Gurobi; HiGHS solves it, and the objective value is returned with no repair. Because we use `gpt-oss-120b` instead of the paper's models, B0 reproduces the setting, not the paper's numbers; Phase 2 compares B0 with the paper's reported SIR results class by class where they exist. Section 3.2 reports the initial B0 results.

B1, SIR with repair. As B0, plus at most r repair rounds ($r = 1$ in the main runs) driven only by the solver status or error text, a simpler form of the self-correction loop of LLMOPT [8]. B0 needs no separate runs, since its answer is the first attempt of B1.

B2, single agent at equal budget (for Gap 3). B1 plus repeated independent samples (temperature 0.7) until its mean token use matches the pipeline's. It answers with the most frequent optimum, and its agreement rate is used as the score in the same calibrated decision rule. B2 is a strong single-agent design at the same cost.

All baselines use the same model, solver, data and grading rule as the pipeline. Grading follows the base paper's released evaluation code: an answer is correct when it is within 0.2% of the reference optimum. The paper text states a tolerance of 10^{-3}, so we will also report results at that stricter level.

2.11 Research Questions and Hypotheses

Table 4 states one narrow empirical question and one testable prediction for each gap.

Table 4: Research questions and hypotheses, one pair per gap.

Gap	Research question	Hypothesis (testable prediction)
Gap 1	**RQ1.** Do requirement-level signals (number coverage, requirement coverage and paraphrase-optimum agreement) detect wrong optimization models better than solver feasibility alone?	**H1.** Adding the signals to feasibility raises the AUROC for separating correct from wrong answers by at least 0.10 on C^3-Bench, and removing any one signal lowers it.
Gap 2	**RQ2.** Does a threshold calibrated on nine problem classes keep the wrong-accept rate at or below the target on the unseen tenth class?	**H2.** With $\alpha = 10\%$, the pooled held-out wrong-accept rate across the ten folds is at most α (reported with its exact 95% interval), while at least 50% of all problems are still answered.
Gap 3	**RQ3.** At the same token budget, does the multi-agent pipeline give a better accuracy-risk trade-off than a single agent?	**H3.** At the largest matched budget ($k = 3$), the pipeline has a lower wrong-accept rate than B2 at equal coverage (50%), because its checks are independent of the formalizer's reasoning. H3 holds only if the 95% bootstrap interval of the paired difference at $k = 3$ lies entirely below zero; the other levels are reported descriptively.

2.12 Evaluation Plan

Evaluation sets. The 100-problem stratified canonical sample, the 30 constraint-augmentation pairs, 30 problems from the linguistic split and OpsOR-Bench (Section 2.7). Fixed seeds make every sample reproducible.

Metrics. Accuracy; solved rate; coverage (share of problems answered); selective accuracy (accuracy on answered problems); wrong-accept rate (share of answered problems that are wrong); AUROC (how well the verifier score separates correct from wrong answers) and risk-coverage curves (wrong-accept rate against coverage as the threshold changes); precision and recall of accepted answers; input tokens, output tokens and LLM calls per problem; latency; and the share of failed runs (crashes or rejected requests) as a reliability measure.

Gap-wise experiments. Each experiment compares the proposed solution with the base-paper setting (B0) on the same data and metrics (Table 5).

Ablations. (i) Without number coverage; (ii) without requirement coverage; (iii) without paraphrase-optimum agreement; (iv) without the parser, formalizing from the question alone; (v) $k = 1, 2, 3$ paraphrases; (vi) fixed against calibrated threshold; (vii) $r = 0, 1, 2$ repair rounds (the main runs use $r = 1$, so the $r = 0$ and $r = 1$ arms come from the main records on the same 30 problems). Ablations (i), (ii), (iii), (v) and (vi) reuse saved run records and need no new LLM calls; lower k is obtained by dropping paraphrase runs, whose tokens and calls the records store. Ablations (iv) and (vii) need new runs (for (vii), only for problems that still fail after one repair) on a 30-problem subset (3 per class). Ablations (i) to (iii) belong to E1, (vi) to E2 and (v) to E3, while (iv) and (vii) show what the parser and the repair rounds add.

Statistics. Exact (Clopper-Pearson) 95% intervals for all rates, McNemar tests for paired accuracy differences, and bootstrap intervals for AUROC and for paired differences in wrong-accept rate. At 50% coverage, tied scores are broken at random and results are averaged over 1,000 draws.

Failure analysis. A sample of wrong answers will be labeled by error type (missing constraint, wrong data value, wrong objective sense, integrality error, code error), together with examples of correct refusals

Table 5: Planned experiments, one or more per gap.

Exp.	Data	Systems compared	Main measurements
E1 Gap 1	Canonical sample; constraint-augmentation pairs; 30 linguistic-split problems	B0, B1, pipeline	AUROC of each signal and of the combined score, against feasibility alone; precision and recall of accepted answers; selective accuracy
E2 Gap 2	Canonical sample in ten leave-one-class-out folds; OpsOR-Bench	B0 (never abstains); pipeline with fixed and with calibrated threshold	Pooled held-out wrong-accept rate with exact 95% interval; per-fold rates (descriptive); coverage; risk-coverage curves
E3 Gap 3	Canonical sample	B0 and B1 (reference points); B2 and the pipeline at $k = 1, 2, 3$	Accuracy; wrong-accept rate at equal coverage (50%) from risk-coverage curves; tokens and calls per problem; McNemar tests and bootstrap intervals; accuracy-cost curves; latency

and false alarms. Failed problems whose reference answer looks doubtful, including every Inventory and Pollution Control failure, will be checked by hand. The Phase 2 results will be summarized in the required gap-resolution table (method or component, base paper result, our result, and whether each gap is fully, partially or not addressed).

2.13 Milestones and Feasibility

Implementation stages. Table 6 follows the official schedule: 10 days for Phase 1, 15 days for Phase 2 and one week for Phase 3. Evaluation starts on Day 1 of Phase 2 and runs alongside development, as the course requires.

Table 6: Milestones and deliverables.

Stage	Time	Tasks and outputs
Phase 1 (done)	10 days	Literature review, proposal, data pipeline, baseline with initial results, first end-to-end pipeline run with the first verifier fixes, repository
Phase 2a	Days 1 to 4	Day 1: fix evaluation samples, add requirement linking to the formalizer prompt, and start B1 runs (which also give B0) and pipeline runs. Days 2 to 4: start the OpsOR-Bench generators and the paper outline
Phase 2b	Days 5 to 8	Compute requirement coverage from saved records; implement and run B2; start the new runs for ablations (iv) and (vii) on 30 problems; finish OpsOR-Bench; build the interactive interface
Phase 2c	Days 9 to 12	E1 with ablations (i) to (iii); E2 leave-one-class-out calibration with ablation (vi) and the OpsOR-Bench shift test; E3 equal-budget comparison with ablation (v); ablations (iv) and (vii); paraphrase check against the linguistic split; draft the paper
Phase 2d	Days 13 to 15	Final E1 to E3 tables on the complete records; failure analysis; gap-resolution table; finish the IEEE-style paper of 6 to 8 pages; Turnitin reports on Day 14; final repository and application
Phase 3	1 week	Live demonstration, slides, rehearsal and viva preparation

Compute and API constraints. The model runs on Amazon Bedrock and the solver on a laptop CPU. Bedrock charges US$0.15 per million input tokens and US$0.60 per million output tokens for this model and has no daily limit at our scale. Our Phase 1 measurements set the budget: the baseline used about 2,800 tokens per problem in one call (mean of both runs), and the pipeline about 18,500 tokens in 6.2 calls on average with $k = 2$ (Section 3.3); with $k = 3$, the pipeline needs about 24,000 tokens in about 8.3 calls.

The full plan (B0, B1, the pipeline with k up to 3, B2, the constraint-augmentation pairs, OpsOR-Bench, two ablations on 30 problems and the paraphrase check on 30 problems of the linguistic split) therefore needs about 8 to 10 million tokens, or about US$4, well within the US$100 credit that AWS gives a new account. At about one minute per pipeline problem, with two problems run at a time, this is about five hours of run time, so the runs fit between Day 1 and Day 13 with room for reruns. B0 is read from the first attempts of B1, B2 reuses its first samples for the smaller budget levels, and lower k and most ablations come from saved records. The fallback if the credits run out or Bedrock is unavailable is in Table 7.

Technical risks and fallback strategy. Table 7 lists the main risks.

Table 7: Technical risks, mitigation and fallback strategy.

Risk	Impact	Mitigation	Fallback
API credits run out, or a provider limits or blocks access	Slower experiments; failed runs	One Bedrock key per member with a budget alert; failed requests retried; saved records reused for ablations, calibration and budget sweeps	Switch to the Groq free plan (200,000 tokens per day and at most 8,000 per minute; the client spaces requests and fits the output cap to that limit), OpenRouter or a local model; then drop OpsOR-Bench runs with $k = 3$ and use $k = 2$ for the constraint-augmentation pairs (main runs keep $k = 3$ and 10 problems per class, so E3 keeps its three budget levels)
Model or provider change	Results not comparable	Exact model ID and settings stored in every run record; one fixed model for all final runs	Re-run the baselines on the replacement model so that comparisons stay paired
Reworded questions change the meaning, or wrong answers agree	Noisy agreement signal	At least two rewordings in the main runs ($k = 1$ is only a lower budget level in E3); comparison with the human-designed linguistic perturbation set	Weight the signal by its measured precision and report it separately
False alarms in number coverage (units, percentages, derived values, index-like integers)	Unneeded escalations	Normalize numbers (percentages, thousands separators and units) and ignore index-like integers such as item numbers; treat number coverage as a soft score	Exclude other known-safe number types and report the false-alarm rate
The new signals do not separate correct from wrong answers (H1 fails)	Weaker refusal decisions	Check the signals early on the Phase 1 and Phase 2a records	Report the negative result; use B2-style sample agreement as the score
B2 does as well as the pipeline (H3 fails)	No measured gain from more agents	B2 stops at the pipeline's mean token budget; achieved ratios are reported	Report the negative result with accuracy-cost curves
Calibrated threshold fails on a held-out class or answers too few problems	Error target missed, or coverage below 50%	Leave-one-class-out design reports every violation and the coverage reached	If the pooled rate exceeds α, report R2 as not met and try a stricter α only post hoc; if only coverage is below 50%, report R2 as partly met
Unsafe generated code	Harm to the host machine	Separate process, time limit, minimal environment, no API keys	Run the solver-runner inside a container

2.14 Work Division Matrix

Table 8 assigns a lead (L) and supporting members (S) to every task, with joint (J) ownership of runs, analysis and writing. Each member commits their own work to the shared repository, and every member must be able to explain the complete system in the viva.

Table 8: Work division matrix (L = lead, S = support, J = joint).

Task	Mushahid Hussain (23I-0541)	Muhammad Waqas (23I-0569)	Abdul Rauf (23I-0591)
Gap 1: verifier signals, number normalization, requirement coverage, ablations	L	S	S
Gap 2: calibration, leave-one-class-out folds, confidence bounds	L	S	S
Gap 3: token metering analysis, single-agent variant B2, budget matching	S	L	S
Baselines B0 and B1, comparison with the base paper	S	L	S
Tests, documentation and README	S	L	S
Data pipeline and OpsOR-Bench	S	S	L
Pipeline integration and interactive interface	S	S	L
Experiment runs on each member's own API key	J	J	J
Results tables, plots and failure analysis	J	J	J
Paper writing, slides and viva preparation	J	J	J
Turnitin reports and final submission	L	S	S

2.15 Potential Publication Direction

If the results are strong, the work can become a paper on trustworthy LLM-based optimization modeling with four contributions:

1. label-free, requirement-level verification signals for generated optimization models, evaluated on a benchmark with exact reference optima;
2. a refusal layer with an explicit error target, together with a test of whether its error control holds under a shift to unseen problem classes (a negative result here would also be useful, as the AdmitOR results suggest [14]);
3. an equal-budget comparison of single-agent and multi-agent auto-formalization, which the reviewed literature does not provide;
4. OpsOR-Bench, a small open benchmark of operations problems with reference models.

The most realistic first target is a workshop at ICLR 2027 or ICML 2027 on LLM agents or machine learning for optimization. With a larger study and stronger results, a full paper could target CPAIOR (the International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research). Following the course policy, any paper would list the group members, the instructor and the supervisor as co-authors according to their contribution.

3 Technical Foundation and Initial Results

3.1 Phase 1 Technical Requirements

Table 9 maps each Phase 1 technical requirement to its evidence in the project repository, `https://github.com/mushi0541/verified-optimization-agents`.

3.2 Initial Baseline Results

Two baseline runs used `gpt-oss-120b` on Amazon Bedrock in the B0 setting (zero-shot SIR prompt, no repair) on the canonical split, at temperature 0. Run A used 3 problems from each of the 10 classes, and Run B used 1 from each. Tables 10 and 11 show the overall and per-class results. Run A and the pipeline

Table 9: Phase 1 technical requirements and where they are met.

Requirement	Evidence in the repository
Finalized system architecture	Sections 2.8 and 2.9, Figure 2; `docs/architecture.md`
Working data pipeline	`scripts/download_data.py` (pinned upstream commit) and `optverify/data.py` (loading and stratified sampling)
Baseline with initial results	`run_baseline` in `optverify/systems.py`; saved runs in `results/` (Section 3.2)
Model integration and provider setup	`optverify/llm.py`: Amazon Bedrock (main provider), plus Groq, OpenRouter, Google AI Studio and Ollama through OpenAI-compatible endpoints, with request pacing
Tool interfaces	`optverify/solver_tool.py`: PuLP and HiGHS in a separate process with a time limit
State and structured contracts	`optverify/contracts.py`: `Problem`, `Attempt`, `SolveResult`, `VerifierReport`, `Decision`, `TokenUsage`, `RunRecord`
Evaluation data and test suite	C^3-Bench splits; 91 offline tests in `tests/` that need no API key and run on every pull request
End-to-end workflow	`run_pipeline` in `optverify/systems.py`; first run in Section 3.3
Clean repository with history	README with setup and run commands, `.env.example`, `requirements.txt`; history of reviewed pull requests, which can merge only when the tests pass

run (Section 3.3) were repeated after we fixed a bug in the solver-runner: on Windows, code that printed a symbol such as ≤ stopped with an encoding error and was counted as a code error.

Table 10: Initial baseline results (B0 setting).

Metric	Run A	Run B
Classes × problems per class	10 × 3	10 × 1
Accuracy	66.7%	80.0%
Solved rate (optimal solver status)	96.7%	100.0%
Mean input tokens per problem	902	985
Mean output tokens per problem	1,781	1,975
LLM calls per problem	1	1
Mean latency per problem	20.8 s	11.1 s

Table 11: Baseline accuracy per class.

Problem class	Run A	Run B
Bin Packing	100.0%	100.0%
Inventory	0.0%	0.0%
Job-Shop Scheduling	66.7%	100.0%
Knapsack	66.7%	100.0%
Min-Cost Network Flow	100.0%	100.0%
Pollution Control	0.0%	0.0%
Portfolio	66.7%	100.0%
Production	100.0%	100.0%
TSP	66.7%	100.0%
Transportation	100.0%	100.0%

Three observations follow. First, Inventory and Pollution Control each failed in all four attempts (three in Run A and one in Run B). The cause may be a formulation problem in these classes or a wrong reference answer (Section 2.12). Second, the samples are very small: with three problems per class, one problem moves a class result by 33 points. Phase 2 therefore uses at least 10 problems per class and reports confidence intervals. Third, 11 of the 39 runs that reached an optimal solver status returned a wrong objective value with no warning. Gaps 1 and 2 address this silent failure. Run records (one JSON object per problem, with the generated code, solver status, tokens and latency) are in `results/`.

3.3 First End-to-End Run of the Proposed Pipeline

To show that the system works end to end, we ran the full pipeline on one canonical problem per class, with the same model at temperature 0. The run used the parser, formalizer, solver-runner (one repair round, $r = 1$), verifier (two paraphrase runs, $k = 2$, each retried once if it returns no optimum), decision layer (fixed threshold of 0.8 in Phase 1) and explainer. Tables 12 and 13 report the results.

Table 12: First end-to-end run of the proposed pipeline (`gpt-oss-120b` on Bedrock, $k = 2$, $r = 1$).

Measure	Value
Problems sampled and completed (one per class, the same as Run B)	10
Answered / escalated / abstained	7 / 3 / 0
Coverage (share answered)	70.0%
Selective accuracy (correct among answered)	100.0% (7 of 7)
Wrong-accept rate (wrong among answered)	0.0% (0 of 7)
Accuracy of the final model, before the decision	80.0% (8 of 10)
Mean input / output tokens per problem	7,847 / 10,649
Mean LLM calls per problem	6.2
Mean latency per problem	67.2 s

Table 13: Verifier signals and decisions for each problem.

Class	Solver status	Number coverage	Paraphrase-optimum agreement	Score	Decision	Model correct
Bin Packing	optimal	1.00	1.00	1.00	answer	yes
Job-Shop Scheduling	optimal	1.00	1.00	1.00	answer	yes
Knapsack	optimal	1.00	1.00	1.00	answer	yes
Min-Cost Network Flow	optimal	1.00	1.00	1.00	answer	yes
Portfolio	optimal	1.00	1.00	1.00	answer	yes
Transportation	optimal	1.00	1.00	1.00	answer	yes
TSP	optimal	1.00	1.00	1.00	answer	yes
Production	optimal	1.00	0.50	0.75	escalate	yes
Inventory	optimal	1.00	0.00	0.50	escalate	no
Pollution Control	optimal	1.00	0.50	0.75	escalate	no

Four early observations follow. They come from only ten problems, so they are not statistical claims. They show that the components work together and what to improve next.

1. **No wrong answer was accepted.** The two wrong models (Inventory and Pollution Control) were escalated because their reworded versions reached different optima, and all seven answers were correct. On the same ten problems, B0 (Run B) returned wrong values for both with no warning, because it never refuses and returns the objective whenever the solver status is optimal.
2. **Retrying failed reworded runs prevents false alarms, but one remains.** One of the 20 reworded runs (TSP) returned no optimum at first, and the retry recovered it; without the retry, the correct TSP answer would have been escalated. The one false alarm is Production: the model was correct, but one rewording returned the same value with the opposite sign (−1,908.17 instead of 1,908.17), which happens when a maximization is written as the minimization of the negative. Phase 2 will treat a value and its negative as the same optimum.
3. **Number coverage needed two fixes.** Item labels such as "Package 3" are not counted as data, and commas in code are read only as separators between values. In an earlier run, TSP code listed distances as tuples such as `(3,14,100.2)`, and the old reader took `14,100.2` for one number with a thousands separator, so 43 distances looked unused. With both fixes, all ten problems reached full coverage.
4. **Verification is not free.** On the same ten problems, the pipeline used about six times the tokens of the baseline (18,500 against 2,960 per problem). A fair comparison with a single agent must therefore match token budgets, which is the purpose of Gap 3.

References

[1] Y. Chen, C. Cheng, Y. Sun, Z. Ling, and D. Ge, "OPT-Engine: Benchmarking the limits of LLMs in optimization modeling via complexity scaling," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, Seoul, South Korea, Jul. 2026, vol. 306, pp. 13807–13834.

[2] H. Liu *et al.*, "Opt-Verifier: Unleashing the power of LLMs for optimization modeling via dual-side verification," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, Seoul, South Korea, Jul. 2026, vol. 306, pp. 78808–78849.

[3] Z. Li *et al.*, "Constructing industrial-scale optimization modeling benchmark," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, Seoul, South Korea, Jul. 2026, vol. 306, pp. 68993–69036.

[4] S. Abdul Rahman, S. A. Andrade Cuellar, G. Raissov, and M. Raza, "VeriSimpl: Robust optimization modeling from natural language using simplification-based verification," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, Seoul, South Korea, Jul. 2026, vol. 306, pp. 103091–103123.

[5] H. Liu, J. Wang, Y. Cai, X. Han, Y. Kuang, and J. Hao, "OptiTree: Hierarchical thoughts generation with tree search for LLM optimization modeling," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, San Diego, CA, USA, Dec. 2025, vol. 38, pp. 120713–120781.

[6] Y. Chen, J. Xia, S. Shao, D. Ge, and Y. Ye, "Solver-informed RL: Grounding large language models for authentic optimization modeling," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, San Diego, CA, USA, Dec. 2025, vol. 38, pp. 106027–106069.

[7] H. Lu, Z. Xie, Y. Wu, C. Ren, Y. Chen, and Z. Wen, "OptMATH: A scalable bidirectional data synthesis framework for optimization modeling," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, Vancouver, BC, Canada, Jul. 2025, vol. 267, pp. 40769–40802.

[8] C. Jiang *et al.*, "LLMOPT: Learning to define and solve general optimization problems from scratch," in *Proc. 13th Int. Conf. Learn. Represent. (ICLR)*, Singapore, Apr. 2025, pp. 101580–101606.

[9] Z. Yang *et al.*, "OptiBench meets ReSocratic: Measure and improve LLMs for optimization modeling," in *Proc. 13th Int. Conf. Learn. Represent. (ICLR)*, Singapore, Apr. 2025, pp. 24726–24759.

[10] C. Huang *et al.*, "ORLM: A customizable framework in training large models for automated optimization modeling," *Oper. Res.*, vol. 73, no. 6, pp. 2986–3009, Nov. 2025, doi: 10.1287/opre.2024.1233.

[11] X. Zhang *et al.*, "OptiMind: Teaching LLMs to think like optimization experts," 2025, arXiv:2509.22979.

[12] J. J. Lian, Y. Sun, H. Chen, C. Zhang, H. Qin, and C.-P. Teo, "ReLoop: Structured modeling and behavioral verification for reliable LLM-based optimization," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2026, to be published, arXiv:2602.15983.

[13] H. Li and M. Hai, "Falsification-based verification of LLM-generated optimization models: Sound test batteries and their detection limits," 2026, arXiv:2607.16646.

[14] J. J. Lian, H. Chen, H. Qin, and C.-P. Teo, "Admission without answers: Label-free certification and experience learning for LLM-based optimization modeling," 2026, arXiv:2608.15565.